Modèles, migrations et Eloquent

Stocker et lire des produits sans écrire de SQL

Séquence
SQ01

Partie
P01P02

Niveau
Mastère ESI

Objectifs

- Créer un modèle et sa migration, puis exécuter la migration
- Insérer, lire et filtrer des données avec Eloquent dans Tinker
- Distinguer la **structure** de la base et les **données** qu'elle contient

Prérequis : projet Laravel installé, base configurée dans `.env`, bases de SQL.

ATTENTION

Le problème

Une boutique veut afficher ses produits : libellé, stock, prix.

Les données doivent **survivre** au redémarrage du serveur, et le schéma doit être **le même** sur chaque poste de l'équipe.

Où vivent les produits, et comment PHP les lit-il ?

Modèle, table, ORM

- La **table** `produits` conserve les lignes en base
- Le **modèle** `Produit` est une classe PHP ; un objet `Produit` = une ligne
- **Eloquent** est l'ORM de Laravel : il traduit les appels PHP en SQL
- Convention : modèle au **singulier**, table au **pluriel**

`Produit::all()` devient `select * from produits`.

Créer le modèle et sa migration

```
php artisan make:model Produit -m
```

Fichier créé	Rôle
`app/Models/Produit.php`	La classe qui manipule les produits
`database/migrations/…_create_produits_table.php`	La création de la table, versionnée

`-m` demande la migration ; le préfixe du fichier est horodaté.

Décrire la table

```
Schema::create('produits', function (Blueprint $table) {
    $table->id();
    $table->string('libelle');
    $table->integer('stock');
    $table->double('prix');
    $table->timestamps();
});
```

Dans la méthode `up()` ; `down()` annule la création.

Choisir le type de colonne

Méthode	Colonne créée
`id()`	Clé primaire auto-incrémentée
`string('libelle')`	Texte court, 255 caractères par défaut
`integer('stock')`	Entier
`double('prix')`	Flottant : approximatif, à éviter pour de l'argent
`decimal('prix', 10, 2)`	Décimal exact : 10 chiffres, dont 2 après la virgule
`timestamps()`	`created_at` et `updated_at`, tenus à jour par Eloquent

Exécuter la migration

```
php artisan migrate
php artisan migrate:status
```

- `migrate` exécute les migrations **en attente**, puis les marque comme faites
- `migrate:status` montre ce qui a été exécuté
- **Créer le fichier ne crée pas la table** : il faut migrer

ATTENTION

Une migration exécutée ne se modifie plus

- Modifier le fichier d'une migration déjà passée **ne change pas** la base
- Sur un projet partagé, on écrit une **nouvelle migration** qui fait évoluer le schéma
- `migrate:fresh` supprime toutes les tables : **les données sont perdues**

Une migration est un commit du schéma : on ajoute, on ne réécrit pas l'histoire.

Autoriser l'insertion groupée

```
class Produit extends Model
{
    protected $fillable = ['libelle', 'stock', 'prix'];
}
```

- `create([...])` remplit plusieurs champs d'un coup : seuls ceux de la liste sont acceptés
- Ce n'est **pas une validation** : un stock de –3 passe
- Laravel 13 propose aussi l'attribut `#[Fillable([...])]`, vu avec le modèle `Voyage`

Insérer dans Tinker

```
php artisan tinker
```

```
use App\Models\Produit;

$produit = Produit::create([
    'libelle' => 'Clavier', 'stock' => 12, 'prix' => 49.90,
]);
$produit->id;
```

`create()` enregistre **une nouvelle ligne** à chaque appel et renvoie l'objet créé.

Lire et filtrer

Expression	Résultat
`Produit::all()`	Une collection de tous les produits
`Produit::find($produit->id)`	Le produit de cet identifiant, ou `null`
`Produit::where('stock', '>', 5)->get()`	Les produits de stock strictement supérieur à 5

`where()` prépare le filtre, `get()` exécute la requête.

Modifier et supprimer

```
$produit = Produit::find($produit->id);

$produit->update(['prix' => 44.90]);

$produit->delete();
```

Avec `create`, `all`, `find`, `update` et `delete` : le **CRUD** complet, sans SQL.

Vérifier, pas seulement regarder

```
$relu = Produit::find($produit->id);

$relu !== null && $relu->libelle === 'Clavier'
    && (int) $relu->stock === 12;
```

Doit afficher `true`. La vérification **relit la base**, elle ne crée aucune ligne.

À RETENIR

À retenir

- **Migration** = structure versionnée · **modèle** = accès aux lignes
- Créer la migration ne crée pas la table : `php artisan migrate`
- `$fillable` autorise des champs, il ne **valide** rien
- Eloquent : `create`, `all`, `find`, `where()->get()`, `update`, `delete`

Ensuite : afficher les produits dans une page web.